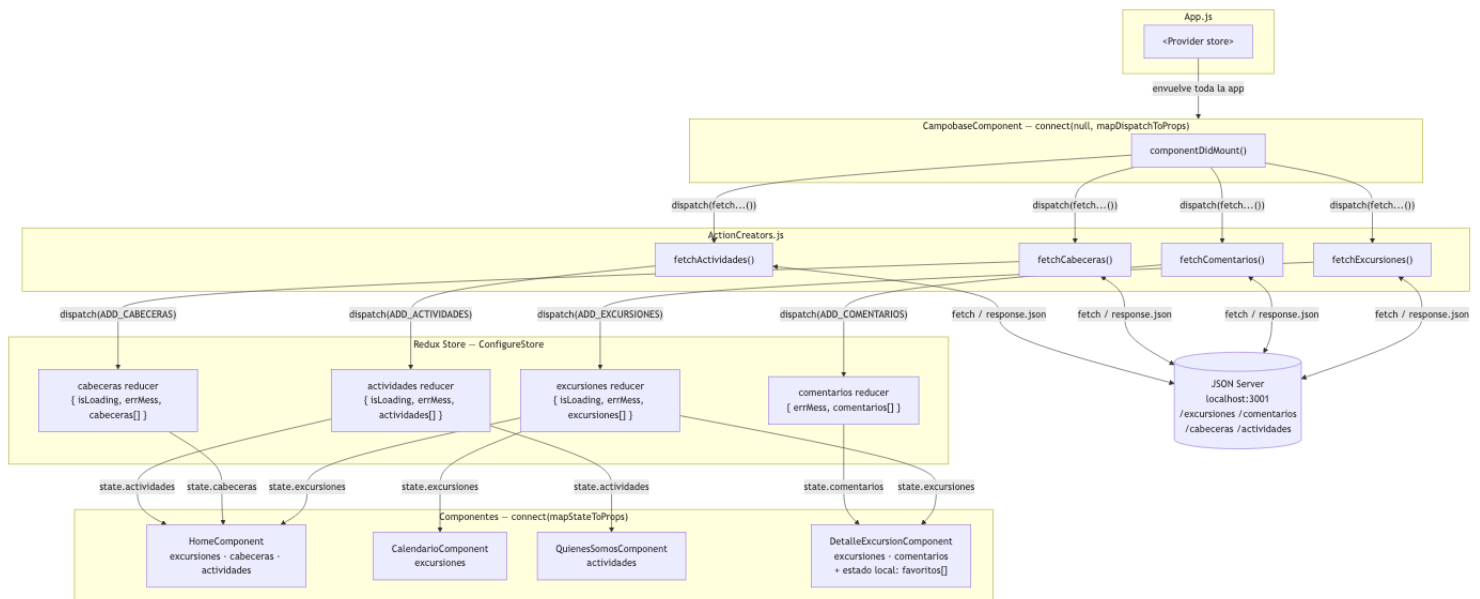


Documento explicativo Redux y Thunk

El objetivo de este ejercicio es refactorizar la aplicación Gaztaroa para adoptar una arquitectura Redux, sustituyendo la gestión de estado local de cada componente por un repositorio de estado centralizado accesible desde cualquier punto de la aplicación.

Antes de la refactorización, cada componente mantenía su propio estado usando `this.state` y los datos se importaban directamente desde ficheros estáticos JavaScript (carpeta `comun/``). Tras la refactorización, los datos se obtienen de una REST API (JSON Server) y se almacenan en el store Redux, al que los componentes se conectan para acceder a ellos. El siguiente diagrama trata de explicar la estructura y funcionamiento de esta nueva arquitectura:



La arquitectura Redux se organiza en torno a cuatro elementos principales:

1. App.js (`redux/configureStore.js`)

El store es el repositorio central de estado. Se crea mediante ``ConfigureStore()`` usando ``@reduxjs/toolkit`` y combina cuatro reducers independientes:

Store:

$$\vdash \text{excursiones} \rightarrow \{ \text{isLoading}, \text{errMess}, \text{excursiones[]} \}$$
$$\vdash \text{comentarios} \rightarrow \{ \text{errMess}, \text{comentarios[]} \}$$

└─ cabeceras → { isLoading, errMsg, cabeceras[] }

└─ actividades → { isLoading, errMsg, actividades[] }

2. CampoBaseComponents

Conectado al store mediante ``connect(null, mapDispatchToProps)``, es el responsable de disparar las cuatro peticiones a la API en cuanto arranca la aplicación, a través de ``componentDidMount()``. No lee estado del store, solo despacha acciones.

3. ActionCreators.js

Son funciones thunk (asíncronas) que realizan las peticiones HTTP a la REST API y despachan las acciones correspondientes al store. Cada `fetchXxx()`:

1. Despacha `LOADING` para indicar que la carga ha comenzado
2. Realiza el `fetch` al servidor JSON (`baseUrl + endpoint`)
3. Si tiene éxito, despacha `ADD_XXX` con los datos recibidos
4. Si falla, despacha `FAILED` con el mensaje de error

Los Action Types usados son constantes definidas en `ActionTypes.js`:

- `EXCURSIONES_LOADING`, `ADD_EXCURSIONES`, `EXCURSIONES_FAILED`
- `ADD_COMENTARIOS`, `COMENTARIOS_FAILED`
- `CABECERAS_LOADING`, `ADD_CABECERAS`, `CABECERAS_FAILED`
- `ACTIVIDADES_LOADING`, `ADD_ACTIVIDADES`, `ACTIVIDADES_FAILED`

4. Reducers (`redux/excursiones.js`, `comentarios.js`, `cabeceras.js`, `actividades.js`)

Cada reducer es una función pura que recibe el estado actual y una acción, y devuelve el nuevo estado. Gestionan tres situaciones: cargando (`LOADING`), datos recibidos (`ADD_XXX`) y error (`FAILED`). Al recibir `ADD_XXX`, almacenan el array de datos en el store y lo ponen disponible para los componentes.

5. Componentes conectados — Consumidores del estado

Los componentes se suscriben al store mediante `connect(mapStateToProps)`. Cada vez que el store se actualiza, Redux re-renderiza automáticamente los componentes afectados con los nuevos datos como `props`. Cada componente declara en `mapStateToProps` qué partes del estado necesita:

- | | |
|--|--|
| Componente | Estado que consume |
| <code>HomeComponent</code> | <code>state.excursiones</code> , <code>state.cabeceras</code> , <code>state.actividades</code> |
| <code>CalendarioComponent</code> | <code>state.excursiones</code> |
| <code>QuienesSomosComponent</code> | <code>state.actividades</code> |
| <code>DetalleExcursionComponent</code> | <code>state.excursiones</code> , <code>state.comentarios</code> |

Flujo completo:

1. App arranca
2. `ConfigureStore()` crea el store (estado inicial: arrays vacíos, `isLoading: true`)

3. `<Provider store={store}>` hace el store accesible a toda la app
4. `CampobaseComponent` se monta $\rightarrow$ `componentDidMount()`
5. `dispatch(fetchExcursiones())` `dispatch(fetchComentarios())`
`dispatch(fetchCabeceras())` `dispatch(fetchActividades())`
6. Cada `ActionCreator` despacha `LOADING` y lanza `fetch()` al JSON Server
7. JSON Server responde con los datos en formato JSON
8. `ActionCreator` despacha `ADD_XXX` con los datos como payload
9. El reducer correspondiente actualiza el store con los nuevos datos
10. Redux detecta el cambio de estado
11. Los componentes conectados via `connect(mapStateToProps)` se re-renderizan
12. Los datos se muestran en pantalla